

COE328 Lab 6 Report

Design of a Simple General-Purpose Processor

Ayman Hasan

DISCLAIMER: This is a reference copy of the report made ONLY for viewing.
Copying and plagiarism is NOT allowed.

1 Table of Contents

1 Table of Contents	2
2 Introduction	2
3 Components	3
Latch 1 and 2	3
4:16 Decoder	4
Finite State Machine (FSM)	6
4 Arithmetic Logic Unit (ALU) 1	7
Description	7
Block Schematic File (BDF)	7
Inputs and Outputs	7
Microcode	7
Waveform	8
5 Arithmetic Logic Unit (ALU) 2	8
Description	8
Block Schematic File (BDF)	9
Inputs and Outputs	9
Microcode	9
Waveform	10
6 Arithmetic Logic Unit (ALU) 3	10
Description	10
Block Schematic File (BDF)	10
Inputs and Outputs	10
Microcode	11
Waveform	11
7 Conclusion	12

2 Introduction

This lab focuses on designing a simplified processor system with the concepts we learnt throughout the course and lab sessions. The system was constructed with several subsystems within Quartus 13.0 using VHDL. The main objective of this lab was to design, simulate, and implement an Arithmetic Logic Unit, otherwise known as ALU, along with the supporting components.

This processor includes registers, a control unit with an FSM and decoder, and the ALU which performs the main logic and calculations. This system takes 2 8-bit inputs, A and B, which are based off of our student numbers, which are processed within the system. We were also given a personalized FSM for which it cycled through different states in a select pattern. The decoder

takes the FSM states into microcode that sends a signal to the ALU which then produces an 8-bit output based on the microcode.

The final design integrates all of the components mentioned before along with a display from the FPGA board, which then can be viewed and tested with. We've also included the modified ALUs which this lab requires of us to add additional features along with their respective waveforms to verify the components.

Values used based on Student Number: A = 31, B = 24

3 Components

Latch 1 and 2

Description

The latch, also known as the register, stores an 8-bit value which is derived from our student numbers. It stores the data on the rising-edge of the clock and outputs it on the next rising-edge of the clock cycle.

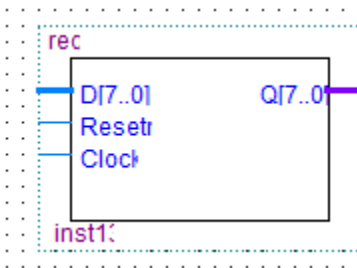
There are 2 latches for each 8-bit value, A and B. They both share the same structure with the only difference being the values.

Truth Table

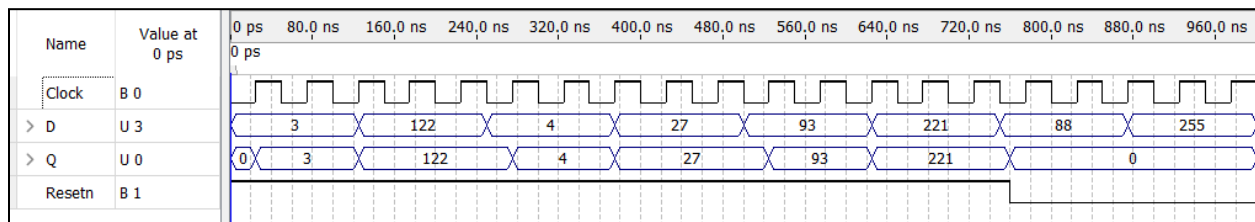
Reset	Clock	Digits of student number, D[0:7]	Storage on next cycle, Q[0:7]	Explanation
0	X	X	0000 0000	Reset clears all data instantly regardless of clock cycle (when Reset is 0 in code)
1	Rising edge	D[0:7]	D[0:7]	On the rising edge, the register read the student number. On the next cycle, the register stores the digits.
	Not rising edge	X	Q[0:7]	Q will continue storing the last values it had in the previous cycle.

Table 3.1 Truth table for the registers.

Diagram



Waveform



4:16 Decoder

Description

This 4:16 decoder is an expansion of the 3:8 decoder. This is done by utilizing the enable as an input. The decoder receives a 4-bit state value from the FSM and converts it into a 16-bit microcode. This microcode has only 1 bit on at a time. Each of these states of the 16-bit output correspond with a specific ALU operation.

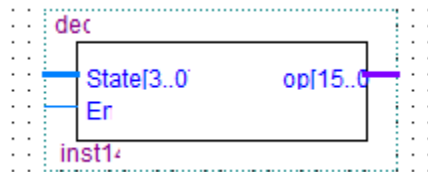
Truth Table

Enable	State, [0:4]	Operation, [0,15]	Explanation
0	XXXX	0000 0000 0000 0000	When the enabler is off, no operation can be selected
1	0000	0000 0000 0000 0001	Each value corresponds with a specific microcode, up to 9 codes
	0001	0000 0000 0000 0010	
	0010	0000 0000 0000 0100	
	0011	0000 0000 0000 0101	

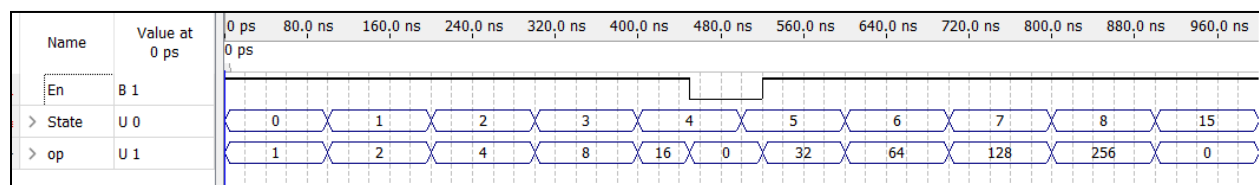
	0011	0000 0000 0000 1000	
	0100	0000 0000 0001 0000	
	0101	0000 0000 0010 0000	
	0110	0000 0000 0100 0000	
	0111	0000 0000 1000 0000	
	1000	0000 0001 0000 0000	
	>1000	0000 0000 0000 0000	
	When the state goes above 9 values, there's no more operations left, therefore the default operation is 0.		

Table 3.2 Truth table of the decoder with explanations

Diagram



Waveform



*Microcode is converted to decimal in waveform to fit easily in screenshot

Finite State Machine (FSM)

Description

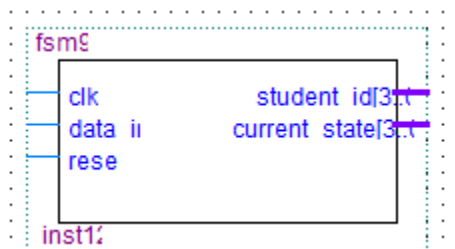
We are reusing the FSM we made in lab 5. This FSM cycles through 9 different states in a specific pattern, unique to the group. It uses a clock as an input and outputs the current state which is used by the decoder for determining the microcode.

Truth Table

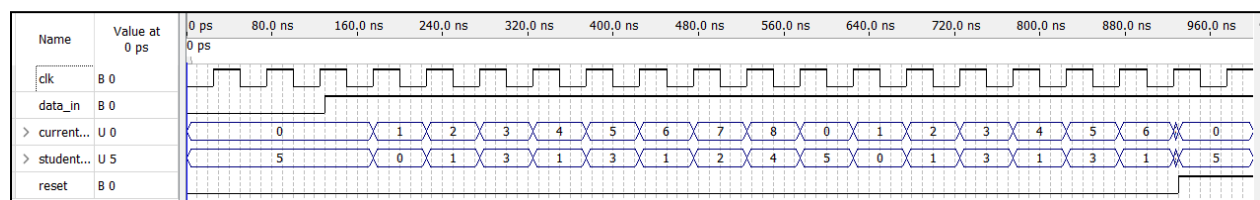
Reset	Data in	Clock	Current State, [0:3]	Student id, [0:3]	Explanation
Rising Edge	X	X	0	5	When reset is switched on, regardless of clock or data enable, everything will reset to state = 0
Not Rising Edge	0	X	Previous state	Previous student id	When data enable is not on, the previous state will be output
	1	Rising Edge	0,1,2,3,4,5,6,7,8 then cycle	5,0,1,3,1,3,1,2,4 then cycle	Every clock cycle, the state will increase by 1 and the student id will display the digits in order

Table 3.3 Truth table for FSM with up counter characteristics

Diagram



Waveform



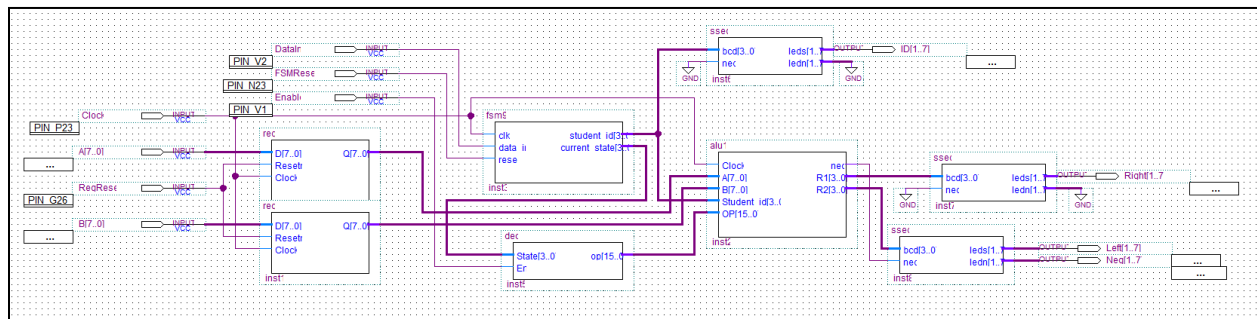
4 Arithmetic Logic Unit (ALU) 1

Description

ALU 1 implements the 9 operations that were given in the lab manual. This includes sum, difference, NAND, etc. The ALU receives 2 8-bit inputs from the register and a 16-bit signal from the control unit. Based on the microcode signal, an operation is performed. The ALU outputs to the seven segment display when it is connected to the FPGA board.

A = 31 and B = 24 were used.

Block Schematic File (BDF)



Inputs and Outputs

Inputs	Outputs
• Clock • A - This is the first 8-bit value • B - This is the second 8-bit value • OP - The 16-bit selector from the control unit • Student_id - not used inside ALU1	• R1 - The right 4 bits of result going to the sseg • R2 - The left 4 bits of result going to the sseg • Neg - Negative sign to sseg

Microcode

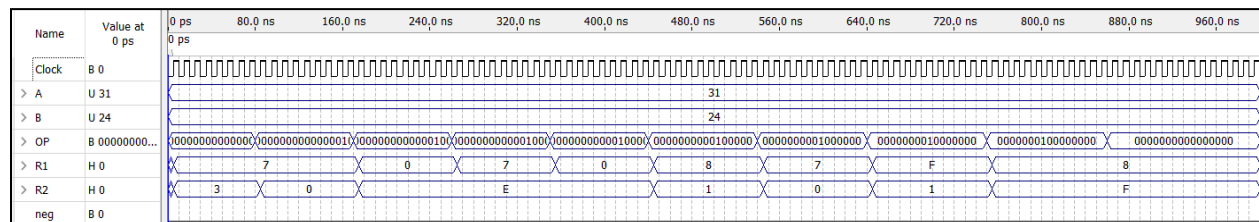
A = 31, B = 24

Function #	Microcode	Operation
1	0000 0000 0000 0001	SUM(A, B)

2	0000 0000 0000 0010	DIFF(A, B)
3	0000 0000 0000 0100	NOT A
4	0000 0000 0000 1000	NOT (A AND B)
5	0000 0000 0001 0000	NOT (A OR B)
6	0000 0000 0010 0000	A AND B
7	0000 0000 0100 0000	A XOR B
8	0000 0000 1000 0000	A OR B
9	0000 0001 0000 0000	A XNOR B

Table 4.1 Microcode for operation selection for ALU 1 as given by Table 1 of the lab manual

Waveform

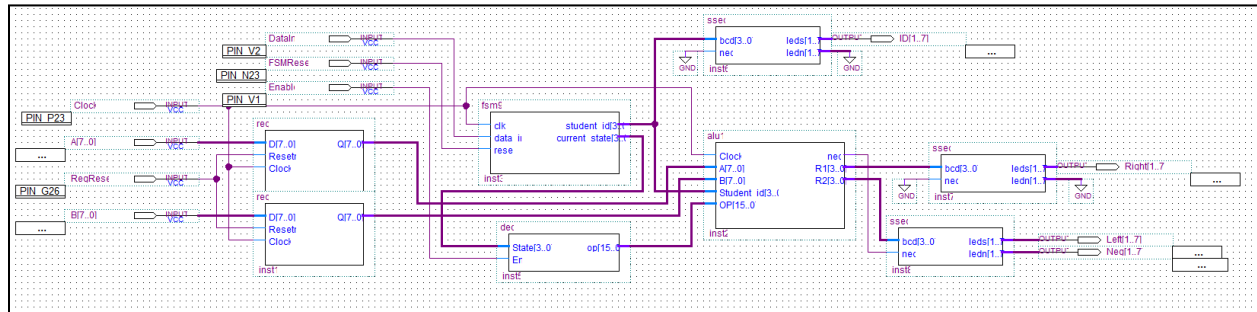


5 Arithmetic Logic Unit (ALU) 2

Description

ALU 2 is a modified version of ALU 1 in the sense that it has 9 new operators specifically taken from the h-set. The general structure of the ALU remains unchanged as it still takes the 2 8-bit inputs, A and B, and the 16-bit operator. Each operator corresponds to a new operation given by the h-set. Only 1 microcode instruction is active at a time.

Block Schematic File (BDF)



Inputs and Outputs

Inputs	Outputs
• Clock • A - This is the first 8-bit value • B - This is the second 8-bit value • OP - The 16-bit selector from the control unit • Student_id - not used inside ALU2	• R1 - The right 4 bits of result going to the sseg • R2 - The left 4 bits of result going to the sseg • Neg - Negative sign to sseg

Microcode

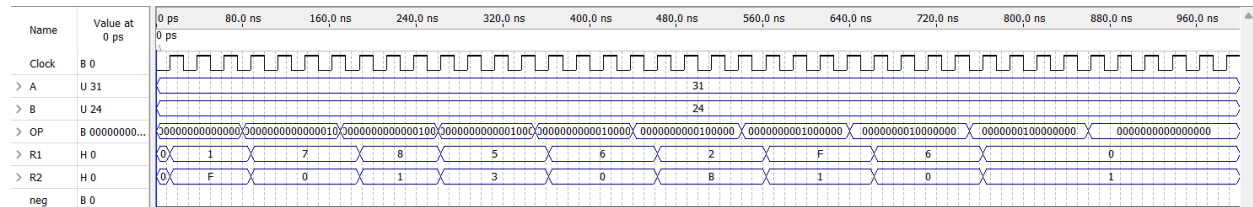
A = 31, B = 24

Function #	Microcode	Operation
1	0000 0000 0000 0001	ROR (A, 4)
2	0000 0000 0000 0010	A XOR B
3	0000 0000 0000 0100	Invert the bit-significance order of B
4	0000 0000 0000 1000	SUM (A, B) - 2
5	0000 0000 0001 0000	ROL (B, 2)
6	0000 0000 0010 0000	Invert the even bits of B
7	0000 0000 0100 0000	Swap the lower 4 bits of B with lower 4 bits of A
8	0000 0000 1000 0000	Shift B to right by 2 bits, input bit = 0

		(SHR)
9	0000 0001 0000 0000	Invert lower four bits of A

Table 5.1 Microcode for operation selection for ALU 2 as given by section 8 part set-h in the lab manual

Waveform

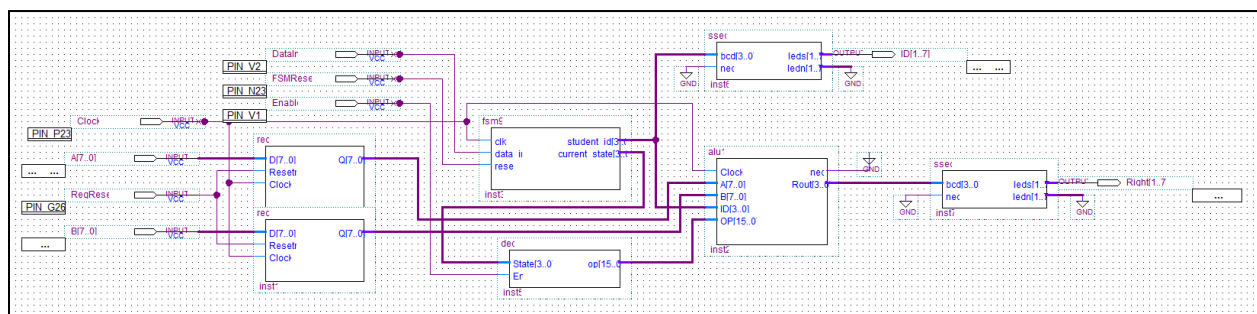


6 Arithmetic Logic Unit (ALU) 3

Description

ALU 3 changes the structure of the unit to only provide YES or NO signals to the sseg. From the assigned h-set, it is meant to compare each of the 2 decimal digits of the B input to the current state's student ID. If any of the digits are greater than the ID, it sends YES, otherwise it sends NO.

Block Schematic File (BDF)



Inputs and Outputs

Inputs	Outputs
• Clock	• Rout - Only sends yes or no to sseg

• A - This is the first 8-bit value • B - This is the second 8-bit value • OP - The 16-bit selector from the control unit • Student_id - Each decimal digit of B compared to current states ID	(1111 = y, 0000 = n) • Neg - Not used in ALU3
---	--

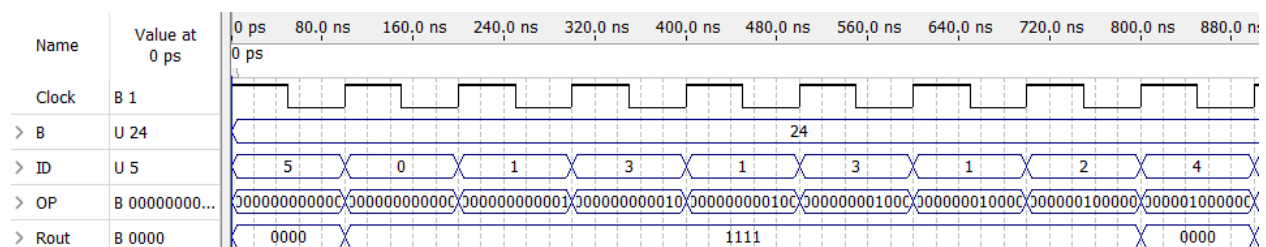
Microcode

A = 31, B = 24, ID = 501313124

Function #	Microcode	Operation
1	0000 0000 0000 0001	Yes if (2 or 4) > ID, else No
2	0000 0000 0000 0010	Yes if (2 or 4) > ID, else No
3	0000 0000 0000 0100	Yes if (2 or 4) > ID, else No
4	0000 0000 0000 1000	Yes if (2 or 4) > ID, else No
5	0000 0000 0001 0000	Yes if (2 or 4) > ID, else No
6	0000 0000 0010 0000	Yes if (2 or 4) > ID, else No
7	0000 0000 0100 0000	Yes if (2 or 4) > ID, else No
8	0000 0000 1000 0000	Yes if (2 or 4) > ID, else No
9	0000 0001 0000 0000	Yes if (2 or 4) > ID, else No

Table 6.1 Microcode for operation selection for ALU 3 as given by section 8 part set-h in the lab manual

Waveform



7 Conclusion

This lab demonstrated the complete design and simulation process of a simple processor in VHDL. This was done by including memory elements, control logic, arithmetic/logical processing, and display handling. By constructing and integrating register units, a finite state machine, a decoder, and ALU cores, a functional GPU-like architecture was developed. Simulation on the waveforms and FPGA verified the behavior of all components and ensured proper microcode-controlled operation sequencing.

The modified ALU and control unit designs further reinforced understanding of extensible digital architecture. Overall, the lab provided hands-on experience with hierarchical design, modular construction, and FPGA simulation.